CSS Flexbox and Responsive Layout

Line things up and make your page work on phones.

Web Dev | Beginner to Intermediate | Ages 10–16

InfiBee Coders

The Big Idea

- Flexbox is the easiest way to line items up in a row or column and space them nicely.
- Put `display: flex;` on the parent (the container) and its children become flex items.
- Responsive design means your page looks good on phones, tablets and computers.
- Flexbox plus a media query solves most layout problems.

Container Properties

- `display: flex;` turns flexbox on.
- `flex-direction: row;` (default) or `column;` sets the direction.
- `justify-content` spaces items along the main direction: `center`, `space-between`, `space-around`, `flex-end`.
- `align-items` lines items up the other way: `center`, `flex-start`, `stretch`.
- `gap: 16px;` adds space between items.
- `flex-wrap: wrap;` lets items move to the next line when there is no room.

Item Properties

- `flex: 1;` makes an item share the free space equally.
- `flex: 2;` makes an item twice as wide as a `flex: 1` item.
- `align-self` changes the position of one item.
- `order` changes the visual order.

Responsive Basics

- Add `<meta name="viewport" content="width=device-width, initial-scale=1">` in the head.
- Use `max-width` and `%` instead of fixed widths.
- A media query changes styles on small screens: `@media (max-width: 600px) { ... }`.
- Test by making the browser window narrow.

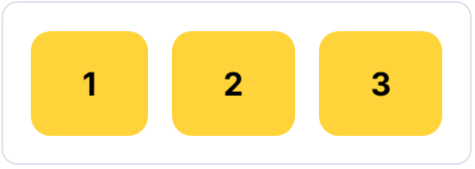
💻 Code Examples

A row of boxes — HTML + CSS

```
<div class="row">
  <div class="box">1</div>
  <div class="box">2</div>
  <div class="box">3</div>
</div>

.row { display: flex; gap: 12px; }
.box { background: #ffd43b; padding: 16px
26px; border-radius: 10px; font-weight: bold;
}
```

▶ What you see:



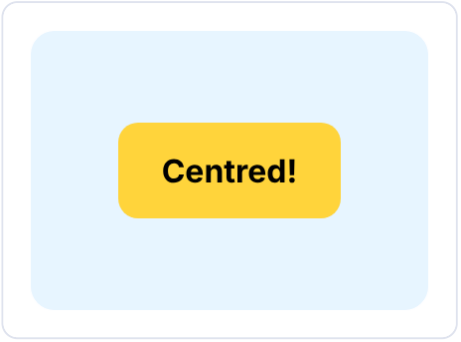
The three boxes sit side by side with space between them.

Centre everything — HTML + CSS

```
<div class="stage">
  <div class="box">Centred!</div>
</div>

.stage {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 140px;
  background: #e7f5ff;
  border-radius: 12px;
}
.box { background: #ffd43b; padding: 14px
22px; border-radius: 10px; font-weight: bold;
}
```

▶ What you see:



justify-content centres across, align-items centres up and down.

A menu bar — HTML + CSS

```
<nav class="menu">
  <a href="#">Home</a>
  <a href="#">About</a>
  <a href="#">Contact</a>
</nav>

.menu {
  display: flex;
  justify-content: space-between;
  background: #1a1a1a;
  padding: 12px 20px;
  border-radius: 10px;
}
.menu a { color: #ffd43b; text-decoration:
none; font-weight: bold; }
```

▶ What you see:



space-between pushes the first item to the left and the last to the right.

Three columns that share the space — HTML + CSS

```
<div class="cols">
  <div class="a">flex: 1</div>
  <div class="b">flex: 2</div>
  <div class="a">flex: 1</div>
</div>

.cols { display: flex; gap: 8px; }
.cols div { padding: 16px; border-radius: 8px;
text-align: center; font-weight: bold; }
.a { flex: 1; background: #b2f2bb; }
.b { flex: 2; background: #ffec99; }
```

▶ What you see:



The middle column is twice as wide as each side column.

A media query — CSS

```
body { font-size: 18px; }
.row { display: flex; }

@media (max-width: 600px) {
  body { font-size: 16px; }
  .row { flex-direction: column; }
}
```

▶ What happens:

On wide screens the boxes sit in a row. On screens 600px wide or smaller the row becomes a column and the text gets smaller.

📚 Key Terms

Flexbox
A layout tool for rows and columns.

Container
The parent element that holds flex items.

Flex item
A child inside a flex container.

Main axis
The direction items are laid out.

justify-content
Spacing along the main axis.

align-items
Alignment across the main axis.

gap
The space between items.

Responsive
Looks good on every screen size.

Media query
A rule that applies at certain screen sizes.

Viewport
The visible area of a web page.

🧮 Quick Reference

```
display: flex;
flex-direction: row | column;
justify-content: center;
align-items: center;
gap: 16px;
flex-wrap: wrap;
flex: 1;
@media (max-width: 600px) { }
```

✏️ Worked Examples

Q: Put three boxes side by side with space between them.
A: Wrap them in a container and give it display: flex; gap: 20px;. The boxes line up in a row.

Q: Centre a box in the middle of its container.
A: On the container use display: flex; justify-content: center; align-items: center; and give it a height.

Q: Stack the boxes on phones.
A: Add @media (max-width: 600px) { .row { flex-direction: column; } }.

🚫 Common Mistakes

- Putting `display: flex` on the items instead of the parent container.
- Centring up and down without giving the container a height.
- Forgetting the viewport meta tag, so phones show a tiny page.

- Using fixed pixel widths that are wider than a phone screen.
- Mixing up `justify-content` (main direction) and `align-items` (cross direction).

👨‍👩‍👧 Parent Corner

- Ask your child to open their page, resize the window and watch what changes.
- Compare flexbox to arranging books on a shelf: you decide spacing and alignment.
- Play Flexbox Froggy, a free online game that teaches these properties.
- Challenge: build a menu bar that becomes a column on a phone.
- Ask them to explain justify-content and align-items in their own words.

🎯 Quick Check

1. Which CSS turns on flexbox?

✅ display: flex; on the parent container.

2. Which property spaces items along the main direction?

✅ justify-content.

3. What does gap do?

✅ Adds space between flex items.

4. What does a media query do?

✅ It applies styles only at certain screen sizes.

5. What does flex: 2 mean?

✅ The item takes twice as much space as an item with flex: 1.